```
// conftest.py
import os
import platform
import re
import time
from datetime import datetime

import pytest
import selenium
from selenium import webdriver
from selenium.webdriver.chrome.service import Service as ChromeService
from selenium.webdriver.edge.service import Service as EdgeService

from mock_api import install_mock
from weather_page import BASE_URL, WeatherPage

BRAVE_PATHS = [
    r"C:\Program Files\BraveSoftware\Brave-Browser\Application\brave.exe",
    r"C:\Program Files (x86)\BraveSoftware\Brave-
Browser\Application\brave.exe",
    "/Applications/Brave Browser.app/Contents/MacOS/Brave Browser",
    "/usr/bin/brave-browser",
]

RESULTS = {}
META = {}


def build_driver():
    browser = os.getenv("BROWSER", "edge").strip().lower()
    binary = os.getenv("BROWSER_PATH", "").strip()
    driver_path = os.getenv("DRIVER_PATH", "").strip()
    if browser == "edge":
        options = webdriver.EdgeOptions()
        factory, service_class = webdriver.Edge, EdgeService
    else:
        options = webdriver.ChromeOptions()
        factory, service_class = webdriver.Chrome, ChromeService
    if browser == "brave" and not binary:
        binary = next((p for p in BRAVE_PATHS if os.path.exists(p)), "")
        if not binary:
            raise RuntimeError("Brave not found. Set BROWSER_PATH to the
full path of brave.exe")
    if binary:
        options.binary_location = binary
    if os.getenv("HEADLESS", "0").strip() == "1":
        options.add_argument("--headless=new")
    if os.getenv("NO_SANDBOX", "0").strip() == "1":
        options.add_argument("--no-sandbox")
        options.add_argument("--disable-dev-shm-usage")
    options.add_argument("--window-size=1366,900")
    options.add_argument("--lang=en-US")
    service = service_class(executable_path=driver_path) if driver_path
else service_class()
```

```
    return factory(options=options, service=service)


@pytest.fixture
def driver():
    drv = build_driver()
    drv.implicitly_wait(0)
    if "browser" not in META:
        caps = drv.capabilities
        name = os.getenv("BROWSER", "edge").strip().lower()
        META["browser"] = f"{name.capitalize()} (engine:
{caps.get('browserName', '')} {caps.get('browserVersion', '')})"
    yield drv
    drv.quit()


@pytest.fixture
def make_page(driver):
    def _make(mock=True, **mock_options):
        if mock:
            install_mock(driver, **mock_options)
        page = WeatherPage(driver, BASE_URL)
        page.open()
        return page

    return _make


def pytest_sessionstart(session):
    META["started_at"] = datetime.now()
    META["clock"] = time.perf_counter()


@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    report = outcome.get_result()
    if report.when == "call" or (report.when == "setup" and report.outcome
!= "passed"):
        longrepr = report.longrepr
        if report.outcome == "skipped" and isinstance(longrepr, tuple):
            message = str(longrepr[2]).replace("Skipped: ", "")
        else:
            message = getattr(getattr(longrepr, "reprcrash", None),
"message", "") or ""
        message = message.strip().splitlines()[0] if message.strip() else
""
        message =
re.sub(r"^(selenium\.common\.exceptions\.)?(TimeoutException: Message:
|AssertionError: |[A-Za-z]+Exception: Message: )", "", message)[:240]
        RESULTS[item.name] = {
            "name": item.name,
            "outcome": report.outcome,
            "duration": report.duration,
```

```
            "message": message,
            "screenshot": None,
        }
    if report.when != "call" or not report.failed:
        return
    drv = item.funcargs.get("driver")
    if drv is None:
        return
    try:
        folder = os.path.join(str(item.config.rootpath), "reports",
"screenshots")
        os.makedirs(folder, exist_ok=True)
        path = os.path.join(folder, re.sub(r"[^A-Za-z0-9_.-]", "_",
item.name) + ".png")
        drv.save_screenshot(path)
        RESULTS[item.name]["screenshot"] = path
        from pytest_html import extras
        report.extras = getattr(report, "extras", []) +
[extras.png(drv.get_screenshot_as_base64())]
    except Exception:
        pass


def pytest_sessionfinish(session, exitstatus):
    if not RESULTS:
        return
    try:
        import make_pdf_report
    except ImportError:
        print("\nPDF report skipped: run  pip install reportlab  to enable
it.")
        return
    started = META.get("started_at", datetime.now())
    seconds = time.perf_counter() - META.get("clock", time.perf_counter())
    url = BASE_URL
    if "localhost" in url or "127.0.0.1" in url:
        note = "Run against a local copy of the application code. The
weather services were replaced with fixed test data."
    else:
        note = "Tests marked live used the real site and services. All
other tests used fixed data in place of the weather services."
    meta = {
        "app_url": url,
        "browser": META.get("browser", ""),
        "os": platform.platform(),
        "tools": f"Python {platform.python_version()} / Selenium
{selenium.__version__} / pytest {pytest.__version__}",
        "started": started.strftime("%d-%b-%Y %H:%M"),
        "duration": f"{seconds / 60:.1f} minutes" if seconds >= 120 else
f"{seconds:.0f} seconds",
        "note": note,
    }
    out = os.path.join(str(session.config.rootpath), "reports",
"Test_Report.pdf")
```

```
    try:
        make_pdf_report.build(list(RESULTS.values()), meta, out)
        print(f"\nPDF test report: {out}")
    except Exception as error:
        print(f"\nPDF report could not be created: {error}")


//mock_api.py
import json
from datetime import date, timedelta

DEFAULT_CITIES = {
    "delhi": [{"name": "Delhi", "latitude": 28.65, "longitude": 77.23,
"country": "IN", "state": "Delhi"}],
    "london": [{"name": "London", "latitude": 51.5, "longitude": -0.12,
"country": "GB", "state": "England"}],
    "mumbai": [{"name": "Mumbai", "latitude": 19.07, "longitude": 72.87,
"country": "IN", "state": "Maharashtra"}],
    "singapore": [{"name": "Singapore", "latitude": 1.29, "longitude":
103.85, "country": "SG"}],
}

REVERSE_NAME = "Connaught Place, New Delhi, Delhi, India"


def build_weather(is_day=1, temp=31.4, humidity=62, wind=12.6, cloud=45,
pressure=1008.2, start="2026-10-08"):
    first = date.fromisoformat(start)
    days = [(first + timedelta(days=i)).isoformat() for i in range(7)]
    hours = range(168)
    return {
        "current": {
            "is_day": is_day,
            "pressure_msl": pressure,
            "temperature_2m": temp,
            "wind_speed_10m": wind,
            "cloud_cover": cloud,
            "relative_humidity_2m": humidity,
        },
        "hourly": {
            "temperature_2m": [round(20 + 0.5 * (h % 24), 1) for h in
hours],
            "relative_humidity_2m": [40 + (h % 24) for h in hours],
            "wind_speed_10m": [round(5 + 0.3 * (h % 24), 1) for h in
hours],
            "cloud_cover": [4 * (h % 24) for h in hours],
        },
        "daily": {
            "time": days,
            "temperature_2m_max": [round(34.1 - 0.3 * i, 1) for i in
range(7)],
            "temperature_2m_min": [round(24.2 - 0.2 * i, 1) for i in
range(7)],
            "sunrise": [f"{d}T06:12" for d in days],
```

```
            "sunset": [f"{d}T18:03" for d in days],
            "wind_speed_10m_max": [15.2] * 7,
            "weather_code": [1] * 7,
        },
    }


MOCK_JS = """
(() => {
  const CFG = __CFG__;
  const realFetch = window.fetch.bind(window);
  const reply = (body, status) => new Response(JSON.stringify(body), {
    status: status || 200,
    headers: {"Content-Type": "application/json"}
  });
  window.__fetchLog = [];
  window.fetch = (input, init) => {
    const url = typeof input === "string" ? input : input.url;
    const handlers = [
      ["api-ninjas.com/v1/geocoding", () => {
        if (CFG.geocode_status !== 200) return reply({error: "mock
failure"}, CFG.geocode_status);
        const city = (new URL(url).searchParams.get("city") || "").trim();
        if (city === "") return reply({error: "city required"}, 400);
        return reply(CFG.cities[city.toLowerCase()] || []);
      }],
      ["open-meteo.com", () => reply(CFG.weather_status === 200 ?
CFG.weather : {error: "mock failure"}, CFG.weather_status)],
      ["nominatim.openstreetmap.org", () => reply({display_name:
CFG.reverse_name})]
    ];
    const match = handlers.find(([key]) => url.includes(key));
    if (!match) return realFetch(input, init);
    window.__fetchLog.push(url);
    return new Promise(resolve => setTimeout(() => resolve(match[1]()),
CFG.delay_ms));
  };
  if (CFG.geo) {
    navigator.geolocation.getCurrentPosition = (ok, fail) => setTimeout(()
=> {
      if (CFG.geo === "denied") {
        if (fail) fail({code: 1, message: "User denied Geolocation"});
      } else {
        ok({coords: {latitude: CFG.geo.lat, longitude: CFG.geo.lon}});
      }
    }, 50);
  }
})();
"""


def install_mock(driver, weather=None, cities=None, weather_status=200,
geocode_status=200,
                 geo=None, reverse_name=REVERSE_NAME, delay_ms=0):
```

```
    cfg = {
        "cities": cities if cities is not None else DEFAULT_CITIES,
        "weather": weather if weather is not None else build_weather(),
        "weather_status": weather_status,
        "geocode_status": geocode_status,
        "geo": geo,
        "reverse_name": reverse_name,
        "delay_ms": delay_ms,
    }
    source = MOCK_JS.replace("__CFG__", json.dumps(cfg))
    driver.execute_cdp_cmd("Page.addScriptToEvaluateOnNewDocument",
{"source": source})

//pytest.ini
[pytest]
addopts = -v --html=reports/test_report.html --self-contained-html
testpaths = .
markers =
    smoke: quick sanity checks
    live: runs against the real network and APIs
    regression: functional regression suite
    negative: invalid input and error handling
    bug: checks written to expose known defects (expected to fail until
the defect is fixed)

//test_weather_app.py
import re
import time

import pytest
from selenium.webdriver.common.action_chains import ActionChains
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys

from mock_api import build_weather
from weather_page import BASE_URL, ERROR_TEXT, WeatherPage


def fmt(value):
    return str(int(value)) if float(value).is_integer() else str(value)


def number(text):
    return float(re.search(r"-?\d+(?:\.\d+)?", text).group())


def searched(make_page, city="Delhi", **mock_options):
    page = make_page(**mock_options)
    page.search(city)
    page.wait_result()
    return page


@pytest.mark.smoke
```

```
@pytest.mark.live
def test_tc01_home_page_loads(make_page):
    page = make_page(mock=False)
    assert page.driver.title == "Weather Forecast"
    assert page.value(page.HEADING) == "Weather Forecast"
    assert page.find(page.CITY).get_attribute("placeholder") == "enter the
city name"
    assert page.shown(page.SEARCH)
    assert page.shown(page.LOCATE)
    assert not page.shown(page.PANEL)


@pytest.mark.live
def test_tc02_page_load_time(driver):
    limit = 5.0
    start = time.perf_counter()
    driver.get(BASE_URL)
    elapsed = time.perf_counter() - start
    print(f"\nPage load time: {elapsed:.2f}s (limit {limit}s)")
    assert elapsed <= limit


@pytest.mark.smoke
@pytest.mark.live
def test_tc03_live_search_real_city(make_page):
    page = make_page(mock=False)
    page.search("London")
    page.wait_result(timeout=30)
    assert "London" in page.value(page.PLACE)


@pytest.mark.smoke
@pytest.mark.regression
def test_tc04_search_valid_city_shows_weather(make_page):
    page = searched(make_page, "Delhi")
    assert "Delhi" in page.value(page.PLACE)
    assert page.value(page.CURR_TEMP) == "31.4Â°C"


@pytest.mark.regression
def test_tc05_current_conditions_match_api_data(make_page):
    page = searched(make_page, "Delhi")
    assert page.value(page.HUMIDITY) == "62"
    assert page.value(page.WIND) == "12.6"
    assert page.value(page.CLOUD) == "45"
    assert page.value(page.PRESSURE) == "1008.2"


@pytest.mark.regression
@pytest.mark.parametrize("is_day, image", [(1, "day.png"), (0,
"night.png")])
def test_tc06_day_night_icon(make_page, is_day, image):
    page = searched(make_page, "Delhi",
weather=build_weather(is_day=is_day))
```

```
    assert
page.find(page.WEATHER_ICON).get_attribute("src").endswith(image)


@pytest.mark.regression
@pytest.mark.parametrize("key, heading", [
    ("temperature", "temperature"),
    ("wind", "wind speed"),
    ("cloud", "cloud cover"),
    ("humidity", "relative humidity"),
])
def test_tc07_hourly_buttons_switch_heading(make_page, key, heading):
    page = searched(make_page)
    page.open_hourly(key)
    assert page.stat_heading().lower() == heading


@pytest.mark.regression
def test_tc08_hourly_temperature_pagination(make_page):
    page = searched(make_page)
    page.open_hourly("temperature")

    def window(start):
        return [(f"{h:02d}00", fmt(round(20 + 0.5 * (h % 24), 1)) + "Â°C")
for h in range(start, start + 8)]

    assert page.hourly_rows() == window(0)
    page.click(page.NEXT)
    assert page.hourly_rows() == window(8)
    page.click(page.NEXT)
    assert page.hourly_rows() == window(16)
    page.click(page.NEXT)
    assert page.hourly_rows() == window(16)
    page.click(page.PREV)
    page.click(page.PREV)
    assert page.hourly_rows() == window(0)
    page.click(page.PREV)
    assert page.hourly_rows() == window(0)


@pytest.mark.regression
@pytest.mark.parametrize("key, unit", [("temperature", "Â°C"), ("wind",
"km/h"), ("cloud", "%"), ("humidity", "%")])
def test_tc09_hourly_values_carry_units(make_page, key, unit):
    page = searched(make_page)
    page.open_hourly(key)
    rows = page.hourly_rows()
    assert len(rows) == 8
    assert all(reading.endswith(unit) for _, reading in rows)


@pytest.mark.bug
@pytest.mark.regression
def test_tc10_boundary_buttons_are_disabled(make_page):
```

```
    page = searched(make_page)
    page.open_hourly("temperature")
    prev_button, next_button = page.driver.find_elements(By.CSS_SELECTOR,
".stats_button")
    assert not prev_button.is_enabled(), "Previous button should be
disabled on the first page"
    page.click(page.NEXT)
    page.click(page.NEXT)
    assert not next_button.is_enabled(), "Next button should be disabled
on the last page"


@pytest.mark.regression
def test_tc11_seven_day_forecast(make_page):
    page = searched(make_page)
    assert not page.shown(page.DAILY)
    page.click(page.GET7)
    assert page.shown(page.DAILY)
    cards = page.daily_cards()
    assert len(cards) == 7
    assert len({card["date"] for card in cards}) == 7


@pytest.mark.bug
@pytest.mark.regression
def test_tc12_daily_min_is_not_greater_than_max(make_page):
    page = searched(make_page)
    page.click(page.GET7)
    for card in page.daily_cards():
        low, high = number(card["values"]["Min"]),
number(card["values"]["Max"])
        assert low <= high, f"{card['date']}: Min shows {low} but Max
shows {high}"


@pytest.mark.regression
def test_tc13_sunrise_and_sunset_format(make_page):
    page = searched(make_page)
    page.click(page.GET7)
    assert page.value((By.ID, "sunrise_day1")) == "06:12 hrs"
    assert page.value((By.ID, "sunset_day1")) == "18:03 hrs"


@pytest.mark.negative
def test_tc14_empty_search_shows_error(make_page):
    page = make_page()
    page.search("")
    page.wait_error()


@pytest.mark.negative
@pytest.mark.parametrize("city", ["zzzzqq", "12345", "!!!@@@", "   "])
def test_tc15_invalid_city_shows_error(make_page, city):
    page = make_page()
```

```
    page.search(city)
    page.wait_error()


@pytest.mark.bug
@pytest.mark.negative
def test_tc16_app_recovers_after_error(make_page):
    page = make_page()
    page.search("zzzzqq")
    page.wait_error()
    page.search("Delhi")
    page.wait_for_place("Delhi", timeout=8, message="A valid search after
an error does not show weather any more")


@pytest.mark.regression
def test_tc17_search_history_suggestions(make_page):
    page = searched(make_page, "Delhi")
    page.search("Mumbai")
    page.wait_for_place("Mumbai")
    page.type_city("del")
    items = page.driver.find_elements(*page.HISTORY_ITEMS)
    assert len(items) == 1 and "Delhi" in items[0].text
    ActionChains(page.driver).move_to_element(items[0]).click().perform()
    assert page.find(page.CITY).get_attribute("value") == "Delhi"
    assert page.driver.find_elements(*page.HISTORY_ITEMS) == []


@pytest.mark.regression
def test_tc18_search_history_survives_reload(make_page):
    page = searched(make_page, "Delhi")
    page.driver.refresh()
    page.wait().until(lambda _: page.shown(page.CITY))
    page.type_city("del")
    assert len(page.driver.find_elements(*page.HISTORY_ITEMS)) == 1


@pytest.mark.regression
def test_tc19_search_history_limited_to_five(make_page):
    page = make_page()
    page.driver.execute_script(
        "for (let i = 1; i <= 7; i++) localStorage.setItem('City' + i, "
        "JSON.stringify({loc: 'City' + i, st: 'S', nation: 'N', temp:
20}));"
    )
    page.type_city("city")
    assert len(page.driver.find_elements(*page.HISTORY_ITEMS)) == 5


@pytest.mark.bug
@pytest.mark.regression
def test_tc20_enter_key_starts_search(make_page):
    page = make_page()
    page.find(page.CITY).send_keys("Delhi", Keys.ENTER)
```

```
    page.wait(5).until(lambda _: page.shown(page.PANEL), message="Pressing
Enter in the search box does not start a search")


@pytest.mark.bug
@pytest.mark.regression
def test_tc21_missing_state_is_not_shown_as_undefined(make_page):
    page = searched(make_page, "Singapore")
    assert "undefined" not in page.value(page.PLACE)


@pytest.mark.regression
def test_tc22_current_location_success(make_page):
    page = make_page(geo={"lat": 28.63, "lon": 77.22})
    page.click(page.LOCATE)
    page.wait_for_place("New Delhi", timeout=10)
    assert page.value(page.LOCATE) == "Use my current location"


@pytest.mark.bug
@pytest.mark.regression
def test_tc23_current_location_text_has_no_trailing_commas(make_page):
    page = make_page(geo={"lat": 28.63, "lon": 77.22})
    page.click(page.LOCATE)
    page.wait_for_place("New Delhi", timeout=10)
    assert not page.value(page.PLACE).rstrip().endswith(","),
page.value(page.PLACE)


@pytest.mark.bug
@pytest.mark.negative
def test_tc24_location_permission_denied_recovers(make_page):
    page = make_page(geo="denied")
    page.click(page.LOCATE)
    page.wait(5).until(
        lambda _: page.value(page.LOCATE) == "Use my current location" and
page.shown(page.SEARCH),
        message="After location is denied the button stays on
'Searching..' and the search button stays hidden",
    )


@pytest.mark.bug
@pytest.mark.negative
def test_tc25_forecast_api_failure_shows_error(make_page):
    page = make_page(weather_status=500)
    page.search("Delhi")
    page.wait(5).until(
        lambda _: page.shown(page.PANEL) and ERROR_TEXT in
page.panel_text(),
        message="No error is shown to the user when the forecast service
fails",
    )
```

```
@pytest.mark.regression
@pytest.mark.parametrize("width", [375, 768])
def test_tc26_layout_fits_small_screens(make_page, width):
    page = make_page()
    page.driver.set_window_size(width, 900)
    page.search("Delhi")
    page.wait_result()
    page.click(page.GET7)
    overflowing = page.driver.execute_script(
        """
        const limit = window.innerWidth + 2;
        return ['#search', '#today', '.days', '#hourly_buttons']
          .filter(sel =>
Array.from(document.querySelectorAll(sel)).some(el => {
            const r = el.getBoundingClientRect();
            return r.right > limit || r.left < -2;
          }));
        """
    )
    assert overflowing == [], f"Elements wider than the {width}px screen:
{overflowing}"


@pytest.mark.bug
def test_tc27_no_api_key_hardcoded_in_client_script(make_page):
    page = make_page(mock=False)
    source = page.driver.execute_async_script(
        "const done = arguments[arguments.length - 1];"
        "fetch('source.js').then(r => r.text()).then(done);"
    )
    assert not re.search(r"api[_-]?key\s*[=:]\s*[\"'][A-Za-z0-9+/=_-
]{16,}", source, re.I), \
        "An API key is hard-coded in source.js and visible to every
visitor"


@pytest.mark.bug
def test_tc28_icon_only_buttons_have_accessible_names(make_page):
    page = make_page()
    unnamed = page.driver.execute_script(
        """
        return Array.from(document.querySelectorAll('button'))
          .filter(b => !((b.getAttribute('aria-label') || '').trim()
                         || b.innerText.trim()
                         || (b.querySelector('img') || {}).alt))
          .map(b => b.id || b.className || 'button');
        """
    )
    assert unnamed == [], f"Buttons without an accessible name: {unnamed}"


@pytest.mark.negative
def test_tc29_script_in_search_box_is_not_executed(make_page):
```

```
    page = make_page()
    page.search('<img src=x onerror="window.__xss=1">')
    page.wait_error()
    assert page.driver.execute_script("return window.__xss || null;") is
None

//weather_page.py
import os

from selenium.common.exceptions import NoSuchElementException
from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.support.ui import WebDriverWait

BASE_URL = os.getenv("BASE_URL", "https://weather-puce-
six.vercel.app/").strip()
ERROR_TEXT = "Error in fetching data"

HOURLY_BUTTONS = {
    "temperature": "hourly_temp",
    "wind": "hourly_w_speed",
    "cloud": "hourly_c_cover",
    "humidity": "hourly_humidity",
}


class WeatherPage:
    HEADING = (By.ID, "heading")
    CITY = (By.ID, "city")
    SEARCH = (By.ID, "search_button")
    LOCATE = (By.ID, "curr_loc")
    HISTORY_ITEMS = (By.CSS_SELECTOR, "#history .history")
    PANEL = (By.ID, "today")
    PLACE = (By.ID, "search_val")
    CURR_TEMP = (By.ID, "curr_temp")
    HUMIDITY = (By.ID, "hum_val")
    WIND = (By.ID, "w_speed_val")
    CLOUD = (By.ID, "cloud_val")
    PRESSURE = (By.ID, "pressure_now")
    WEATHER_ICON = (By.CSS_SELECTOR, "#weather_img img")
    MAIN_STATS = (By.ID, "main_stats")
    PREV = (By.ID, "prev")
    NEXT = (By.ID, "next")
    GET7 = (By.ID, "get7")
    DAILY = (By.ID, "daily")

    def __init__(self, driver, url=BASE_URL, timeout=15):
        self.driver = driver
        self.url = url
        self.timeout = timeout

    def wait(self, timeout=None):
        return WebDriverWait(self.driver, timeout or self.timeout)
```

```
    def open(self):
        self.driver.get(self.url)
        self.wait().until(EC.visibility_of_element_located(self.CITY))
        self.wait().until(lambda d: d.execute_script(
            "const h = document.getElementById('head');"
            "return h.getBoundingClientRect().left >= 0 &&
getComputedStyle(h).opacity === '1';"
        ))

    def find(self, locator):
        return self.driver.find_element(*locator)

    def shown(self, locator):
        try:
            return self.find(locator).is_displayed()
        except NoSuchElementException:
            return False

    def value(self, locator):
        return (self.find(locator).get_attribute("textContent") or
"").strip()

    def panel_text(self):
        return self.find(self.PANEL).text

    def click(self, locator):
        element =
self.wait().until(EC.presence_of_element_located(locator))
        self.driver.execute_script("arguments[0].scrollIntoView({block:
'center'});", element)
        self.wait().until(EC.element_to_be_clickable(locator)).click()

    def type_city(self, text):
        field = self.find(self.CITY)
        field.clear()
        if text:
            field.send_keys(text)

    def search(self, city):
        self.type_city(city)
        self.click(self.SEARCH)

    def wait_result(self, timeout=None):
        def ready(_):
            if not self.shown(self.PANEL):
                return False
            if ERROR_TEXT in self.panel_text():
                raise AssertionError(f"Expected weather data but the page
shows '{ERROR_TEXT}'")
            return self.value(self.PLACE) not in ("", "Place")

        self.wait(timeout).until(ready)

    def wait_error(self, timeout=None):
```

```
        self.wait(timeout).until(
            lambda _: self.shown(self.PANEL) and ERROR_TEXT in
self.panel_text(),
            message=f"'{ERROR_TEXT}' was not shown",
        )

    def wait_for_place(self, text, timeout=None, message=None):
        self.wait(timeout).until(lambda _: text in self.value(self.PLACE),
message=message)

    def open_hourly(self, key):
        self.click((By.ID, HOURLY_BUTTONS[key]))
        self.wait().until(EC.visibility_of_element_located(self.MAIN_STATS
))

    def hourly_rows(self):
        rows = []
        for i in range(1, 9):
            time_label = self.value((By.CSS_SELECTOR, f"#stat{i} .time"))
            reading = self.value((By.CSS_SELECTOR, f"#stat{i} > span"))
            rows.append((time_label, reading))
        return rows

    def stat_heading(self):
        return self.value((By.CSS_SELECTOR, "#stat .hourly_parameter"))

    def daily_cards(self):
        cards = []
        for card in self.driver.find_elements(By.CSS_SELECTOR, ".days"):
            texts = lambda css: [
                e.get_attribute("textContent").strip() for e in
card.find_elements(By.CSS_SELECTOR, css)
            ]
            labels = texts(".daily_stat_names p")
            values = texts(".daily_stat_values p")
            cards.append({
                "date": card.find_element(By.CSS_SELECTOR,
".date").get_attribute("textContent").strip(),
                "values": dict(zip(labels, values)),
            })
        return cards
```